

Isolated observability metrics stack — conductor runbook

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Authority:** task #56 enablement — hermetic control-plane /metrics scrape proof **Scope:** deploy/observability/compose.metrics.yml + tests/observability/gen_test_mtls.sh + tests/observability/metrics_scrape_test.sh **Maintainer:** helix_proxy control-plane stream

1. Overview

This runbook boots a **fully-isolated** three-service stack — obs-postgres + obs-redis + obs-api — whose only purpose is to prove the control-plane's plaintext Prometheus /metrics listener serves real exposition, **without** depending on the base stack's unknown Postgres password and **without** the dynamic profile's gluetun/WireGuard operator credentials.

Why a separate stack (FACT, §11.4.6):

- control-plane/cmd/api/main.go:66-75 requires HELIX_PG_DSN and calls store.Open → sql.Open("pgx", dsn) + PingContext. The api **fail-fast aborts** if Postgres is unreachable or authentication fails.
- The committed docker-compose.observability.yml points the api's DSN at the base proxy-postgres, whose password initialises from the operator's helixproxy_pg_password secret on first init of the proxy-pgdata volume — a value we do not know, and an already-initialised data dir will not re-read a new secret. That Postgres also lives in the dynamic profile beside proxy-gluetun, which needs operator WireGuard creds to come up.
- deploy/observability/compose.metrics.yml sidesteps both: a **fresh** isolated volume (obs-pgdata) + the **same** test password secret used **symmetrically** by obs-postgres (initdb POSTGRES_PASSWORD_FILE) and obs-api (runtime DSN), so postgres and api match by construction.

Migration note (FACT): there is **no Go migrator** — no store.Migrate exists (control-plane/internal/store/{store.go,postgres.go}). store.Open only opens + pings. The

schema is applied by Postgres **initdb**: the compose mounts .././sql (repo sql/) at /docker-entrypoint-initdb.d, so on first boot Postgres runs schema.sql then seed_example.sql alphabetically (the sql/migrations/ subdir is ignored by initdb — top-level *.sql only). The schema is **not required** for the metrics exposition itself — the two counters are pre-registered and pre-touched to 0 (control-plane/internal/api/metrics.go:65-68), so valid exposition is served even with no tables — but initdb makes vpnUpCollector clean and surfaces the helix_proxy_vpn_up{profile} series for seeded profiles. No manual migration step is needed.

Host ports (all loopback-pinned; chosen to avoid the in-use set 5432/6379/53128/51080/58080/59090):

Service	Host	Container	Purpose
obs-api	127.0.0.1:59091	59091	plaintext /metrics (the proof)
obs-postgres	127.0.0.1:55432	5432	conductor psql verify (optional)
obs-redis	127.0.0.1:56379	6379	conductor redis-cli verify (optional)

The api's mTLS control port :58080 is **internal** to the obs-metrics network — never published (the host's 58080 is the base proxy-admin whoami).

2. Prerequisites

- Rootless podman + podman-compose (§11.4.161).
- The control-plane image builds (control-plane/Containerfile → api binary), or \${CONTROL_PLANE_IMAGE} already present locally.
- openssl (for the test mTLS material), bash, coreutils.
- The chosen host ports 55432 / 56379 / 59091 free (verify with `ss -ltn | grep -E ':(55432|56379|59091)'` — no output = free).

3. Conductor sequence (boot + scrape)

Run every heavy step under the §12.6 resource cap (GOMAXPROCS=2 nice -n 19 ionice -c 3 ...). All commands run from the **repo root**.

Step 1 — generate the disposable test mTLS material + test pg password

```
GOMAXPROCS=2 nice -n 19 ionice -c 3 \
  bash tests/observability/gen_test_mtls.sh
```

Writes 7 files under the gitignored tests/observability/.mtls/ (\$11.4.10 / \$11.4.30): ca.{key,crt}, server.{key,crt}, client.{key,crt}, pg_password.txt. Idempotent (re-run is a no-op unless --force). It prints the four podman secret create commands (Step 2) — it never prints secret bytes.

Step 2 — create the 4 podman secrets (NAMES fixed by the compose)

```
podman secret create helixproxy_api_cert      tests/
           observability/.mtls/server.crt
podman secret create helixproxy_api_key      tests/
           observability/.mtls/server.key
podman secret create helixproxy_api_client_ca tests/
           observability/.mtls/ca.crt
podman secret create helixproxy_pg_password  tests/
           observability/.mtls/pg_password.txt
```

helixproxy_pg_password is the **throwaway test cred** consumed symmetrically by obs-postgres (initdb) and obs-api (DSN). If a secret already exists: podman secret rm <name> then re-create. (These are the SAME 4 secret names the committed docker-compose.observability.yml uses, so the material is shared with that overlay when both are booted.)

Step 3 — boot the isolated trio (this stack owns its own pg/redis)

```
podman-compose -f deploy/observability/compose.metrics.yml up -d
```

obs-postgres runs initdb (schema + seed) on the fresh obs-pgdata volume; obs-api retries via restart: unless-stopped until obs-postgres/obs-redis are healthy (podman-compose ignores depends_on: service_healthy). Wait for the api healthcheck to go healthy:

```
podman healthcheck run obs-api           # or: podman ps --filter
           name=obs-api
```

Optional sanity: podman logs obs-api should end with
api ...: serving PLAINTEXT /metrics on 0.0.0.0:59091 and serving
mTLS control-API on :58080.

Step 4 — run the anti-bluff /metrics scrape guard against the isolated URL

```
HELIX_OBSERVABILITY_STACK=1 \  
HELIX_METRICS_URL=http://127.0.0.1:59091/metrics \  

```

```
GOMAXPROCS=2 nice -n 19 ionice -c 3 \
bash tests/observability/metrics_scrape_test.sh
```

- `HELIX_OBSERVABILITY_STACK=1` declares the stack up (else the guard SKIPS).
- `HELIX_METRICS_URL` overrides the default `:59090` to the isolated `:59091`.
- The guard asserts real exposition `CONTENT` (`# HELP/# TYPE + helix_proxy_acl_decisions_total + helix_proxy_tunnel_down_responses_total`), never merely `HTTP 200` (§11.4.107).
- The counter-increment sub-proof drives a request through `HELIX_PROXY_URL` (default `http://127.0.0.1:53128`). This isolated stack has **no squid proxy**, so that request is unreachable → the sub-proof records an honest `feature_disabled_by_config SKIP` while the exposition-content proof `PASSes`. That is correct and anti-bluff — the increment path is `P5/P10-pending` (`control-plane/internal/api/metrics.go:14-17`).

Evidence lands under `qa-results/observability/metrics_scrape/<run-id>/` (gitignored).

4. Teardown

```
podman-compose -f deploy/observability/compose.metrics.yml down
-v # -v drops obs-pgdata
# Optional: remove the shared test secrets (only if no other obs
# stack needs them):
# podman secret rm helixproxy_api_cert helixproxy_api_key \
# helixproxy_api_client_ca
# helixproxy_pg_password
```

5. Edge cases / troubleshooting

- **obs-api restart-loops** — `obs-postgres` still in `initdb`, or the `helixproxy_pg_password` secret differs from the one `obs-postgres` initialised with. Because the volume is fresh and the secret is used symmetrically, this only happens if the secret was rotated between Steps 2 and 3. Fix: `down -v` (drop the volume), re-create the secret, `up -d` again.
- **Scrape SKIPS instead of PASS** — `HELIX_OBSERVABILITY_STACK=1` not exported.
- **Scrape FAILs “no valid exposition”** — api not yet healthy; wait for the healthcheck, then re-run. Confirm `curl -s http://127.0.0.1:59091/metrics | head`.
- **Port bind error** — another process took `55432/56379/59091`; override via `OBS_PG_PORT / OBS_REDIS_PORT / OBS_METRICS_PORT` env before `up -d`.

6. Related

- `deploy/observability/compose.metrics.yml` — this stack.
- `docker-compose.observability.yml` — the committed overlay (base-stack coupled).
- `tests/observability/gen_test_mtls.sh` — the test-material generator.
- `tests/observability/metrics_scrape_test.sh` — the §11.4.115 RED/GREEN scrape guard.
- `docs/scripts/gen_test_mtls.md` — companion doc for the generator.